📘 LESSON 3 — VISIBILITY, LAYERS & ARCHITECTURE ENFORCEMENT

Using **Bazel**

🧠 FIRST: WHAT PROBLEM DOES `visibility` SOLVE?

Without visibility (CMake world)

- Any module can include any header
- Anyone can link to anything
- Architecture slowly **rots**
- Refactoring becomes dangerous

Bazel philosophy

> **Dependencies are allowed only if explicitly permitted**

This is **compile-time architecture enforcement**.

🔑 CORE IDEA (VERY IMPORTANT)

> `deps` **says "I want to use you"**
> `visibility` **says "You are allowed to use me"**

Both must agree.

🧩 WHAT IS `visibility`?

`visibility` controls **who can depend on a target**, not who can include headers.

📄 Basic Syntax

```
visibility = ["//visibility:public"]
```

Means:

> "Anyone in the workspace can depend on this target"

🚨 DEFAULT VISIBILITY (MOST IMPORTANT FACT)

If you do **NOT** specify visibility:

```
cc_library(
    name = "math",
)
```

Then visibility is:

```
PRIVATE to this package only
```

➡️ **Even sibling folders cannot use it**

🧠 VISIBILITY ≠ HEADER VISIBILITY (COMMON CONFUSION)

Concept	Controlled by
Who can depend	`visibility`
Which headers are usable	`hdrs`
Who gets headers	`deps`

They are **orthogonal**.

🧩 VISIBILITY TYPES (ALL YOU WILL USE)

1️⃣ `//visibility:public` (USE SPARINGLY)

```
visibility = ["//visibility:public"]
```

Meaning

- Any target anywhere can depend on it

When allowed

- Core utilities
- Stable foundational APIs
- Logging, math, types

When dangerous

- Feature modules
- Business logic
- App-specific code

2️⃣ Package-Private (DEFAULT)

```
# no visibility specified
```

Meaning

- Only targets in **same folder** can depend

Best for

- Internal helpers
- Implementation details
- Private glue code

3️⃣ Subpackage Visibility (MOST IMPORTANT FOR LARGE PROJECTS)

```
visibility = ["//perception:__subpackages__"]
```

Meaning

- Any package under `perception/` can depend

Example allowed:

```
perception/tracking
perception/fusion
```

Example blocked:

```
apps/vehicle_app ❌
```

4️⃣ Exact Package Visibility (STRICT CONTROL)

```
visibility = ["//apps/vehicle_app:__pkg__"]
```

Meaning

- Only **that exact package** can depend

Used for

- Tight contracts
- Experimental modules
- Safety-critical code

🏗️ REAL-WORLD 100-MODULE ARCHITECTURE (THIS IS GOLD)

Define LAYERS FIRST (NON-NEGOTIABLE)

```
core
↓
drivers
↓
perception
↓
planning
↓
control
↓
apps
```

Dependencies must flow downward only.

📁 core/math/BUILD

```
cc_library(
    name = "math",
    hdrs = ["math.h"],
    srcs = ["math.cpp"],
    visibility = ["//visibility:public"],
)
```

✔ Safe
✔ Stable
✔ Foundational

📁 drivers/lidar/BUILD

```
cc_library(
    name = "lidar_driver",
    srcs = ["lidar.cpp"],
    hdrs = ["lidar.h"],
    deps = ["//core/math:math"],
    visibility = [
        "//perception:__subpackages__",
    ],
)
```

💡 Key decision explained

- Apps should NOT talk to drivers directly
- Perception is allowed
- Enforced by Bazel

📁 perception/object_detection/BUILD

```
cc_library(
    name = "object_detection",
    srcs = ["detector.cpp"],
    hdrs = ["detector.h"],
    deps = [
        "//drivers/lidar:lidar_driver",
```

```
    ],
    visibility = [
        "//planning:__subpackages__",
        "//apps:__subpackages__",
    ],
)
```

Architecture is now **encoded in BUILD files**.

🔴 COMMON VISIBILITY ERRORS (AND FIXES)

❌ ERROR 1 — “Target is not visible”

```
Target '//drivers/lidar:lidar_driver' is not visible
```

Root cause

Consumer not listed in `visibility`.

Fix

Add correct visibility rule (not public by default).

❌ ERROR 2 — Everything is public (ARCHITECTURAL SMELL)

Symptom

- No visibility errors ever
- Anyone can depend on anything

Fix

Start locking down:

- Drivers → perception only
- Perception → planning only

❌ ERROR 3 — Over-restricting visibility

```
Target not visible from //perception/...
```

Fix

Use `__subpackages__` instead of `__pkg__`.

🧠 GOLDEN RULES FOR VISIBILITY

1️⃣ Default to **private**
2️⃣ Public is for **foundations only**
3️⃣ Use subpackage visibility for layers
4️⃣ Visibility = architecture, not convenience
5️⃣ If Bazel blocks you → design problem, not Bazel problem

🧠 ONE-LINE MENTAL MODEL

> `deps` **describe usage.** `visibility` **enforces permission. Bazel guarantees architecture.**

✅ WHAT YOU NOW MASTER

✔ Visibility mechanics
✔ Package vs subpackage rules
✔ Layer enforcement
✔ Preventing dependency leaks
✔ Scaling to 100+ modules safely

➡️ NEXT LESSON (VERY IMPORTANT)

👉 **Lesson 4 —** `select()` **, configurations & build variants**

- Debug vs release
- Platform-specific code
- ARM vs x86
- Feature toggles (without `#ifdef` hell)

Say:

> **continue to lesson 4**

You're now thinking like a **Bazel architect**, not a beginner 💪